Table des matières

Exercices corrigés 04 — Fonctions de fenêtrage 1
Questions courtes 1
Questions pratiques 2
Diagnostics 3
Questions à développer 4
Mises en situation 4
Corrigé 6

Exercices corrigés 04 — Fonctions de fenêtrage

Comment travailler ces exercices. Répondez à chaque énoncé seul, puis vérifiez. Les questions courtes vérifient que le vocabulaire est en place ; les pratiques vous font écrire du SQL, tantôt sur le transactionnel, tantôt sur l'entrepôt ; les diagnostics vous montrent du code qui cloche ; les mises en situation vous demandent de choisir une approche et de la défendre. Comptez quatre heures pour y répondre, le corrigé non compris.

Les repères de terrain. Un énoncé marqué `[OLTP]` se traite dans PostgreSQL sur `Adventureworks` (schémas `sales`, `production`). Un énoncé marqué `[OLAP]` se traite dans DuckDB sur `adventureworksdw.duckdb`, ouvert par `USE dbo;`. Sans marque, la question vaut pour les deux.

Le corrigé suit les énoncés, en fin de document. Répondez à l'énoncé en entier avant d'y passer : c'est en butant sur une question qu'on découvre ce qu'on avait lu sans le comprendre.

Questions courtes

1. La différence de fond. En une phrase, qu'est-ce qu'une fonction de fenêtre fait que `GROUP BY` ne fait pas ?
2. Les trois morceaux. Que contient un `OVER (…)` au maximum, et à quoi sert chacun des trois éléments ?
3. `PARTITION BY` contre `GROUP BY`. Les deux « regroupent ». Quelle est la différence sur le nombre de lignes en sortie ?
4. Les trois classements. Sur les prix 100, 90, 90, 80, donnez ce que produisent `ROW_NUMBER`, `RANK` et `DENSE_RANK`.
5. Choisir son classement. Vous voulez garder exactement une ligne par groupe, même en cas d'égalité. Laquelle des trois fonctions choisissez-vous, et pourquoi pas les deux autres ?

© 2026 iAlexMG. Tous droits réservés.

6. `NTILE`. À quoi sert `NTILE(4)` ? Que se passe-t-il si le nombre de lignes n'est pas divisible par 4 ?
7. `LAG` et `LEAD`. Que renvoie `LAG(ventes) OVER (ORDER BY mois)` sur la toute première ligne, et pourquoi ?
8. L'argument oublié. `LAG(ventes, 3, 0)` : que valent le deuxième et le troisième argument ?
9. La fenêtre vide. Que fait `SUM(x) OVER ()`, sans rien entre les parenthèses ?
10. Le cadre par défaut. Quand vous écrivez `SUM(x) OVER (ORDER BY d)`, quel cadre s'applique sans que vous l'ayez demandé ? Et quand il n'y a pas d'`ORDER BY` ?
11. `ROWS` contre `RANGE`. Donnez la différence en une phrase, puis un cas concret où les deux donnent un résultat différent.
12. Le piège du défaut. Pourquoi une moyenne mobile écrite `AVG(x) OVER (ORDER BY d)` sans cadre explicite n'est-elle pas une moyenne mobile ?
13. `QUALIFY`. À quoi sert-il, et sur quel moteur de cette formation existe-t-il ?
14. Pourquoi pas le `WHERE`. Expliquez pourquoi on ne peut pas filtrer sur une fonction de fenêtre directement dans le `WHERE`. Rattachez votre réponse à l'ordre d'exécution du Cours 01.
15. L'équivalent portable. Sans `QUALIFY`, comment obtient-on le même résultat ? Nommez les deux écritures possibles.
16. `[OLAP]` La dimension temps. Pour obtenir l'année et le mois d'une vente sur l'entrepôt, que faites-vous ?
17. `[OLTP]` La même chose sans dimension. Et sur le transactionnel, où vit la date, et quelle fonction la découpe ?

1. Le nombre de jointures. Pour une même question analytique, laquelle des deux voies demande généralement le plus de jointures, et pourquoi est-ce voulu de part et d'autre ?
2. Ce qui ne change pas. Nommez quatre éléments de la syntaxe de fenêtrage qui sont rigoureusement identiques sur les deux moteurs de la formation. Nommez ensuite le seul qui diffère.
3. Le coût caché. Une fonction de fenêtre impose une opération au moteur, laquelle ? Sur PostgreSQL, quel levier permet de la lui épargner, et à quelle condition ce levier sert-il vraiment ? Le même levier vaut-il sur l'entrepôt ?
4. Fenêtre et agrégat ensemble. Peut-on écrire `SUM(SUM(x)) OVER (…)` ? Si oui, que fait chaque niveau ?
5. La part du total. Écrivez, en une ligne et sans contexte de table, l'expression qui donne la part d'une ligne dans la somme de toutes les lignes.

Questions pratiques

Écrivez la requête complète avant de regarder. Quand un énoncé existe en deux versions, faites les deux et comparez-les : c'est là que le contraste entre les deux mondes devient concret.

© 2026 iAlexMG. Tous droits réservés.

P1. [OLAP] Rang par couleur. Pour chaque produit, affichez son nom, sa couleur, son prix et son rang de prix à l'intérieur de sa couleur (le plus cher vaut 1). Ignorez les produits sans couleur et sans prix.

P2. [OLTP] Rang par couleur, l'autre voie. La même chose sur `production.product`. Notez ce qui change et ce qui ne change pas.

P3. [OLAP] Champion par sous-catégorie. Avec `QUALIFY`, ne gardez que le produit le plus cher de chaque sous-catégorie.

P4. [OLTP] Champion par sous-catégorie, sans `QUALIFY`. Le même résultat dans PostgreSQL. Affichez le libellé de la sous-catégorie, pas son numéro, ce qui vous demandera une jointure de plus.

P5. Part du total. Pour chaque produit dont le prix est supérieur à zéro, affichez son prix et sa part dans la somme de tous les prix de catalogue. Donnez la requête pour votre terrain préféré, puis dites ce qui changerait sur l'autre.

P6. [OLAP] Croissance mensuelle. Reprenez les ventes mensuelles et ajoutez une colonne de croissance en pourcentage par rapport au mois précédent. Protégez la division.

P7. [OLTP] Croissance mensuelle, l'autre voie. La même chose, en découpant `orderdate` vous-même.

P8. Les trois premiers mois de chaque année. Pour chaque année, ne gardez que les trois mois aux ventes les plus fortes, classés. Écrivez la version portable, celle qui marche des deux côtés.

P9. Le cumul qui repart. Ajoutez aux ventes mensuelles un cumul qui repart à zéro à chaque nouvelle année, à côté du cumul global. Un seul `OVER` par colonne.

P10. (avancé) L'écart au meilleur. Pour chaque produit, affichez son prix, le prix du produit le plus cher de sa sous-catégorie, et l'écart en pourcentage entre les deux. Une seule passe sur la table, aucune sous-requête d'agrégat.

P11. (avancé) La médiane par groupe. Pour chaque sous-catégorie, donnez le prix médian de ses produits, en vous servant uniquement de fonctions de fenêtrage. Dites ensuite quelle fonction aurait fait ça en une ligne, et pourquoi l'exercice vous demande l'autre chemin.

P12. (avancé) Deux cadres dans une requête. Sur les ventes mensuelles, affichez côte à côte la moyenne des trois derniers mois et la moyenne des trois mois suivants. Dites ce que chaque colonne vaut sur les lignes du bord, combien de lignes sont touchées de chaque côté, et pourquoi.

Diagnostics

On vous montre du code qui ne fait pas ce que son auteur croit. Dites ce qui cloche, pourquoi, et donnez le correctif.

E1. Une analyste veut la moyenne mobile des ventes sur trois mois. Elle écrit :

```sql
SELECT annee, mois, ventes,
       AVG(ventes) OVER (ORDER BY annee, mois) AS moy_mobile_3
FROM mensuel;
```


© 2026 iAlexMG. Tous droits réservés.

Le résultat monte doucement et ne ressemble à aucune moyenne mobile. Expliquez ce que la requête calcule réellement, nommez le mécanisme en cause, et corrigez.

E2. Un développeur veut les trois produits les plus chers par sous-catégorie sur PostgreSQL. Il écrit :

```sql
SELECT name, listprice,
       ROW_NUMBER() OVER (PARTITION BY productsubcategoryid
                          ORDER BY listprice DESC) AS rang
FROM production.product
WHERE rang <= 3;
```

Le moteur répond que la colonne `rang` n'existe pas. Il conclut que PostgreSQL ne gère pas les fonctions de fenêtrage. Expliquez-lui son erreur, dites pourquoi le message est trompeur, et donnez les deux écritures correctes.

E3. Une requête calcule la part de chaque vente dans le total annuel :

```sql
SELECT annee, mois, ventes,
       ventes / SUM(ventes) OVER (ORDER BY annee) AS part_annee
FROM mensuel;
```

Les parts de la première année s'additionnent bien à 1, mais celles des années suivantes s'additionnent à de moins en moins : 0,6846, puis 0,4861, puis 0,1821. Et tous les mois d'une même année sont divisés par le même nombre, ce qui ressemble à ce qu'on voulait sans l'être. Trouvez les deux fautes distinctes, et dites pourquoi la première année, elle, tombe juste.

Questions à développer

Un court paragraphe de raisonnement, pas un mot juste.

A. La fenêtre et le grain. Expliquez pourquoi une fonction de fenêtre ne change jamais le grain du résultat, alors qu'un `GROUP BY` le change toujours. Puis dites en quoi cette propriété est précisément ce qui rend le fenêtrage indispensable au reporting, avec un exemple où l'agrégation classique obligerait à écrire deux requêtes et une jointure.

B. Le même langage, deux terrains. Vous avez fait le laboratoire des deux côtés. Expliquez, à quelqu'un qui n'a vu que l'entrepôt, ce qui l'attend s'il doit écrire la même analyse sur une base transactionnelle. Distinguez ce qui relève de la syntaxe, ce qui relève du schéma, et ce qui relève de la performance.

C. `QUALIFY`, confort ou piège. `QUALIFY` rend le code plus court et plus lisible, et il tourne sur l'entrepôt de cette formation. Défendez d'abord son usage, puis donnez l'argument sérieux qu'on peut lui opposer dans une équipe qui travaille sur plusieurs moteurs. Expliquez au passage pourquoi la ligne de partage n'est pas celle qu'on croit, entre gros moteur et petit moteur. Concluez par la règle que vous adopteriez.

Mises en situation

© 2026 iAlexMG. Tous droits réservés.

On décrit une entreprise et une question métier. À vous de choisir l'approche, de l'écrire, et surtout de la justifier. Il y a plusieurs bonnes réponses, mais il y a des réponses indéfendables.

M1. Épicerie Bellefeuille, le palmarès qui bouge. Une chaîne de douze épiceries veut un écran hebdomadaire qui montre, pour chaque succursale, ses cinq produits les plus vendus de la semaine, avec leur rang, leur part dans le chiffre de la succursale, et leur progression de rang par rapport à la semaine précédente. Les données vivent dans le système de caisse, une base transactionnelle PostgreSQL, avec une table de tickets et une table de lignes de ticket.

Décrivez la requête que vous écririez : quelles fenêtres, sur quelles partitions, dans quel ordre, et combien de passes sur les données. La progression de rang est le point difficile : dites pourquoi, et comment vous la traitez. Puis répondez à la question qui décide de tout : cet écran doit-il être servi par la base transactionnelle ou par un entrepôt, et qu'est-ce qui tranche ?

M2. Clinique vétérinaire Deux-Montagnes, l'intervalle entre deux visites. La clinique veut savoir, pour chaque animal, le délai entre deux consultations consécutives, afin de repérer les patients qu'on ne revoit plus. Elle veut aussi le délai moyen par espèce, et la liste des animaux dont la dernière visite remonte à plus du double de leur délai habituel.

Écrivez le raisonnement complet : quelles fonctions de fenêtrage, dans quel ordre, et combien de niveaux de requête. Précisez ce que vous faites de la toute première visite de chaque animal, qui n'a pas de précédente, et de ceux qui n'ont jamais eu qu'une seule visite. Terminez en disant pourquoi la dernière demande, celle du double du délai habituel, ne peut pas se traiter dans la même fenêtre que les deux premières.

M3. (avancé) Coopérative forestière Abitibi, le contraste imposé. La coopérative a deux systèmes. Le système d'exploitation, transactionnel, contient les livraisons de bois au jour le jour depuis quatre ans, soit 40 millions de lignes. L'entrepôt, alimenté chaque nuit, contient les mêmes livraisons modélisées en étoile. On vous demande le même rapport dans les deux systèmes : la part de chaque coupe dans le volume mensuel de sa région, avec un cumul annuel glissant.

Écrivez les deux requêtes, ou décrivez-les précisément si vous préférez. Puis comparez-les sur quatre axes : le nombre de jointures, la lisibilité, la fraîcheur du résultat, et le risque pour le système. Dites laquelle vous mettriez en production et dans quelles conditions vous changeriez d'avis. Un des quatre axes joue contre l'entrepôt : identifiez-le et ne l'escamotez pas.

© 2026 iAlexMG. Tous droits réservés.

Corrigé

La numérotation est celle des énoncés.

Questions courtes

1. Elle calcule une valeur agrégée sans écraser les lignes : chaque ligne d'entrée reste en sortie, avec la valeur calculée en face d'elle.

Théorie : `Theorie_04_SQL.md`, section 1.

2. Au maximum trois éléments. `PARTITION BY` découpe en groupes indépendants, `ORDER BY` ordonne les lignes à l'intérieur de chaque groupe, et le cadre (`ROWS` ou `RANGE`) limite les lignes prises en compte autour de la ligne courante.

Théorie : `Theorie_04_SQL.md`, section 2.

3. `GROUP BY` réduit : n lignes en entrée, une ligne par groupe en sortie. `PARTITION BY` ne réduit rien : n lignes en entrée, n lignes en sortie.

Théorie : `Theorie_04_SQL.md`, sections 1 et 2.

4. Sur 100, 90, 90, 80 : `ROW_NUMBER` donne 1, 2, 3, 4 ; `RANK` donne 1, 2, 2, 4 (il saute) ; `DENSE_RANK` donne 1, 2, 2, 3 (il ne saute pas).

Théorie : `Theorie_04_SQL.md`, section 3.

5. `ROW_NUMBER`, parce qu'elle seule garantit des numéros tous différents, donc `= 1` retourne exactement une ligne. `RANK` et `DENSE_RANK` répètent le rang en cas d'égalité et en retourneraient deux. Attention au corollaire : le choix de la ligne gardée devient arbitraire si l'`ORDER BY` ne départage pas, d'où l'habitude d'ajouter une colonne de départage.

Théorie : `Theorie_04_SQL.md`, section 3.

6. `NTILE(4)` répartit les lignes en quatre paquets de taille aussi égale que possible, les quartiles. Si le compte n'est pas divisible par 4, les premiers paquets reçoivent une ligne de plus que les derniers.

Théorie : `Theorie_04_SQL.md`, section 3.

7. `NULL`, parce qu'il n'y a aucune ligne avant la première dans l'ordre demandé. C'est le comportement voulu, et c'est aussi pourquoi toute soustraction qui s'appuie dessus donne `NULL` sur la première ligne.

Théorie : `Theorie_04_SQL.md`, section 4.

8. Le deuxième argument est le décalage, ici trois lignes en arrière au lieu d'une : trois lignes, et non trois mois, tant qu'un mois vide peut manquer à la série. Le troisième est la valeur de remplacement quand il n'y a pas de ligne à cette distance, ici zéro au lieu de `NULL`, sur les trois premières lignes. Ce zéro se paie sur une variation : ces trois lignes affichent tout leur total comme une hausse, là où `NULL` disait qu'il n'y avait rien à comparer.

© 2026 iAlexMG. Tous droits réservés.

Théorie : `Theorie_04_SQL.md`, section 4.

9. Elle applique la somme à la totalité du jeu de résultats, sans découpage ni ordre. C'est l'écriture du total général, celle qui sert à calculer une part.

Théorie : `Theorie_04_SQL.md`, section 5.

10. Avec `ORDER BY` et sans cadre explicite, le défaut est `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`, donc un cumul depuis le début. Sans `ORDER BY`, le défaut porte sur toute la partition, donc un total répété.

Théorie : `Theorie_04_SQL.md`, section 6.

11. `ROWS` compte des lignes, `RANGE` compte des valeurs de la colonne d'ordre. Avec un décalage, la différence est nette : `ROWS BETWEEN 1 PRECEDING AND CURRENT ROW` prend la ligne courante et celle d'avant, deux lignes au plus, alors que `RANGE BETWEEN 1 PRECEDING AND CURRENT ROW` prend toutes les lignes dont la valeur d'ordre tombe entre la valeur courante moins un et la valeur courante, ex aequo compris. Mesuré sur cinq lignes ordonnées par 1, 2, 2, 4 et 5, qui portent 10, 20, 30, 40 et 50 : sur la ligne 4, la somme vaut 70 en `ROWS`, parce que la ligne d'avant est un 2, et 40 en `RANGE`, parce qu'aucune valeur ne tombe à 3 ; sur les deux 2, `RANGE` somme 60, les deux ex aequo et le 1. Sans décalage, `RANGE` ne désigne plus qu'un groupe : `CURRENT ROW` y veut dire la ligne courante et tous ses ex aequo, ce qui est le cadre par défaut. Mêmes sommes sur nos deux terrains.

Théorie : `Theorie_04_SQL.md`, section 6.

12. Parce que le cadre par défaut est un cumul depuis le début, pas une fenêtre glissante. La colonne obtenue est la moyenne de tout ce qui précède, qui se lisse de plus en plus au fil du temps, et ne dit rien des trois derniers mois. Il faut écrire le cadre : `ROWS BETWEEN 2 PRECEDING AND CURRENT ROW`.

Théorie : `Theorie_04_SQL.md`, section 6.

13. `QUALIFY` filtre sur le résultat d'une fonction de fenêtre, après son calcul. Dans cette formation, il tourne sur DuckDB, donc sur la voie OLAP ; PostgreSQL ne le connaît pas et répond « erreur de syntaxe ».

Théorie : `Theorie_04_SQL.md`, section 7.

14. Parce que le `WHERE` s'exécute avant que les fenêtres soient calculées. L'ordre logique est `FROM`, `WHERE`, `GROUP BY`, `HAVING`, fenêtres, `SELECT`, `ORDER BY` : au moment du `WHERE`, la colonne de rang n'existe pas encore. Ce n'est donc pas une limitation de moteur, c'est une conséquence de l'ordre d'exécution.

Théorie : `Theorie_04_SQL.md`, sections 2 et 7.

15. Deux écritures. Une sous-requête, la fenêtre à l'intérieur et le filtre à l'extérieur. Ou une CTE (`WITH`, Cours 05), qui fait la même chose en plus lisible.

Théorie : `Theorie_04_SQL.md`, section 7.

16. `[OLAP]` On joint le fait à `DimDate` sur la clé de date, et on lit `CalendarYear` et `MonthNumberOfYear`, deux colonnes déjà taillées pour ça.

Théorie : `Theorie_04_SQL.md`, section 8.

© 2026 iAlexMG. Tous droits réservés.

17. [OLTP] La date vit dans `orderdate`, sur l'en-tête de commande. On la découpe avec `EXTRACT` (ou `DATE_TRUNC` selon ce qu'on veut), et il faut penser à typer le résultat.

Théorie : `Theorie_04_SQL.md`, section 8. Théorie : `Theorie_02_SQL.md`, section 6.

18. La voie transactionnelle, parce que le schéma est normalisé : le contexte d'une vente est éclaté dans plusieurs tables qu'il faut rejoindre. C'est voulu des deux côtés, pour des raisons opposées. En OLTP, la normalisation évite la redondance et les anomalies de mise à jour, ce qui compte quand des milliers d'écritures arrivent par seconde. En OLAP, la dénormalisation évite les jointures, ce qui compte quand des requêtes balaient des millions de lignes.

Théorie : `Theorie_04_SQL.md`, section 8.

19. Par exemple : le mot-clé `OVER`, la clause `PARTITION BY`, la clause `ORDER BY` interne, les cadres `ROWS BETWEEN`, et les fonctions `ROW_NUMBER`, `RANK`, `DENSE_RANK`, `LAG`, `LEAD`, `NTILE`. Tout cela est du SQL standard, et les deux labos le vérifient au caractère près. Le seul qui diffère est `QUALIFY`, que DuckDB a et que PostgreSQL n'a pas.

Théorie : `Theorie_04_SQL.md`, section 8.

20. Un tri. Une fenêtre exige d'ordonner les lignes à l'intérieur de chaque partition avant de pouvoir numéroter ou cumuler.

Sur PostgreSQL, le levier est un index qui porte déjà l'ordre demandé, c'est-à-dire les colonnes de partition puis celles d'ordre, dans cet ordre-là. Mesuré sur les 121 317 lignes de `sales.salesorderdetail` : sans index, le plan porte un `Sort` déversé sur disque ; avec, le `Sort` disparaît au profit d'un `Index Scan`. La condition tient au volume, et c'est la seconde partie de la question : la même mesure sur les 504 lignes de `production.product` ne change rien du tout, parce que la table tient en quelques pages et que le planificateur juge le balayage moins cher.

Le même levier ne vaut pas sur l'entrepôt. Un index créé sur DuckDB laisse le plan identique, un balayage suivi de l'opérateur `WINDOW`, et le temps aussi. Ce qui fait la vitesse d'un moteur en colonnes, c'est de ne lire que les colonnes utiles et de trier en mémoire des données déjà compactes, pas de suivre un index. Le principe commun est de ne pas tout trier ; le moyen n'est pas transposable. Une réponse qui prête ce rôle aux micro-partitions de Snowflake se trompe de mécanisme : d'après sa documentation, elles servent à écarter des blocs devant un filtre, et elle ne dit rien d'un tri qu'elles épargneraient.

Théorie : `Theorie_04_SQL.md`, section 8.

21. Oui. Le `SUM` intérieur est un agrégat classique qui s'applique après le `GROUP BY` ; le `SUM` extérieur est une fenêtre qui s'applique au résultat de cette agrégation. C'est l'écriture qui permet, en une seule requête, d'obtenir le total par mois et le cumul de ces totaux.

Théorie : `Theorie_04_SQL.md`, section 5.

22. `x / SUM(x) OVER ()`. Et en pratique on protège la division : `x / NULLIF(SUM(x) OVER (), 0)`.

Théorie : `Theorie_04_SQL.md`, section 5.

Questions pratiques

P1. [OLAP]

© 2026 iAlexMG. Tous droits réservés.

```sql
SELECT EnglishProductName, Color, ListPrice,
       ROW_NUMBER() OVER (PARTITION BY Color ORDER BY ListPrice DESC) AS rang_couleur
FROM dbo.DimProduct
WHERE ListPrice > 0 AND Color IS NOT NULL
ORDER BY Color, rang_couleur;
```

Théorie : `Theorie_04_SQL.md`, section 3.

P2. [OLTP]

```sql
SELECT name, color, listprice,
       ROW_NUMBER() OVER (PARTITION BY color ORDER BY listprice DESC) AS rang_couleur
FROM production.product
WHERE listprice > 0 AND color IS NOT NULL
ORDER BY color, rang_couleur;
```

Ce qui change : le nom de la table, celui des colonnes, et la casse. Ce qui ne change pas : la fenêtre, au caractère près. C'est le constat central du cours, et il vaut la peine de le vérifier soi-même plutôt que de le lire.

Théorie : `Theorie_04_SQL.md`, section 3.

P3. [OLAP]

```sql
SELECT ProductSubcategoryKey, EnglishProductName, ListPrice
FROM dbo.DimProduct
WHERE ListPrice > 0 AND ProductSubcategoryKey IS NOT NULL
QUALIFY ROW_NUMBER() OVER (PARTITION BY ProductSubcategoryKey
                           ORDER BY ListPrice DESC, ProductKey) = 1
ORDER BY ProductSubcategoryKey;
```

Théorie : `Theorie_04_SQL.md`, section 7.

P4. [OLTP]

```sql
SELECT s.name AS sous_categorie, c.name AS produit, c.listprice
FROM (
    SELECT p.name, p.listprice, p.productsubcategoryid,
           ROW_NUMBER() OVER (PARTITION BY p.productsubcategoryid
                              ORDER BY p.listprice DESC) AS rang
    FROM production.product AS p
    WHERE p.listprice > 0 AND p.productsubcategoryid IS NOT NULL
) AS c
JOIN production.productsubcategory AS s
  ON s.productsubcategoryid = c.productsubcategoryid
```


© 2026 iAlexMG. Tous droits réservés.

```sql
WHERE c.rang = 1
ORDER BY sous_categorie;
```

Deux écarts à relever. Le `QUALIFY` devient une sous-requête, ce qui ajoute deux niveaux d'indentation. Et obtenir le libellé de la sous-catégorie demande une jointure que l'entrepôt n'exigeait pas, puisque `DimProduct` porte déjà l'information repliée. C'est la modélisation dimensionnelle qui se paie en amont et se rembourse à chaque requête.

Théorie : `Theorie_04_SQL.md`, sections 7 et 8.

P5.

```sql
-- OLAP
SELECT EnglishProductName, ListPrice,
       ListPrice / NULLIF(SUM(ListPrice) OVER (), 0) AS part_du_total
FROM dbo.DimProduct
WHERE ListPrice > 0
ORDER BY part_du_total DESC;

-- OLTP : seuls les noms changent.
SELECT name, listprice,
       listprice / NULLIF(SUM(listprice) OVER (), 0) AS part_du_total
FROM production.product
WHERE listprice > 0
ORDER BY part_du_total DESC;
```

Rien d'autre ne change, parce que la question ne fait intervenir ni la date ni une table voisine. C'est un bon test : chaque fois qu'une question ne demande qu'une seule table, les deux voies se confondent.

Théorie : `Theorie_04_SQL.md`, sections 7 et 8.

P6. `[OLAP]`

```sql
SELECT annee, mois, ventes,
       (ventes - LAG(ventes) OVER (ORDER BY annee, mois))
         / NULLIF(LAG(ventes) OVER (ORDER BY annee, mois), 0) AS croissance
FROM (
    SELECT d.CalendarYear AS annee, d.MonthNumberOfYear AS mois,
           SUM(f.SalesAmount) AS ventes
    FROM dbo.FactInternetSales AS f
    JOIN dbo.DimDate           AS d ON d.DateKey = f.OrderDateKey
    GROUP BY d.CalendarYear, d.MonthNumberOfYear
) AS mensuel
ORDER BY annee, mois;
```


© 2026 iAlexMG. Tous droits réservés.

Théorie : `Theorie_04_SQL.md`, section 4.

P7. `[OLTP]`

```sql
SELECT annee, mois, ventes,
       (ventes - LAG(ventes) OVER (ORDER BY annee, mois))
         / NULLIF(LAG(ventes) OVER (ORDER BY annee, mois), 0) AS croissance
FROM (
    SELECT EXTRACT(YEAR  FROM h.orderdate)::int AS annee,
           EXTRACT(MONTH FROM h.orderdate)::int AS mois,
           SUM(d.linetotal)                     AS ventes
    FROM sales.salesorderheader AS h
    JOIN sales.salesorderdetail AS d ON d.salesorderid = h.salesorderid
    GROUP BY 1, 2
) AS mensuel
ORDER BY annee, mois;
```

La partie fenêtrée est identique mot pour mot. Toute la différence tient dans la sous-requête qui fabrique les ventes mensuelles : une jointure vers une dimension d'un côté, un `EXTRACT` et une jointure de reconstitution de l'autre.

Théorie : `Theorie_04_SQL.md`, sections 4 et 8.

P8.

```sql
SELECT annee, mois, ventes, rang
FROM (
    SELECT annee, mois, ventes,
           ROW_NUMBER() OVER (PARTITION BY annee ORDER BY ventes DESC) AS rang
    FROM mensuel
) AS classe
WHERE rang <= 3
ORDER BY annee, rang;
```

La version portable est celle sans `QUALIFY`. Elle fonctionne sur les deux moteurs, et c'est l'argument qu'on oppose à `QUALIFY` dans une équipe multi-moteurs (voir la question C).

Attendu sur l'entrepôt : 11 lignes, et non 15. Les ventes en ligne couvrent 38 mois répartis sur cinq années civiles, mais les deux années du bord n'ont qu'un mois chacune, décembre 2010 et janvier 2014. Le classement ne peut pas rendre trois mois là où il n'y en a qu'un, et il ne s'en plaint pas. C'est la même leçon qu'au `QUALIFY` de la théorie : un top N rend au plus N lignes par groupe, et compter sur le compte plein est une hypothèse sur les données, pas une propriété de la requête.

Théorie : `Theorie_04_SQL.md`, sections 3 et 7.

© 2026 iAlexMG. Tous droits réservés.

P9.

```sql
SELECT annee, mois, ventes,
       SUM(ventes) OVER (ORDER BY annee, mois)                  AS cumul_global,
       SUM(ventes) OVER (PARTITION BY annee ORDER BY mois)      AS cumul_annuel
FROM mensuel
ORDER BY annee, mois;
```

Le `PARTITION BY annee` est tout ce qui sépare les deux colonnes. C'est l'exemple le plus net de ce que fait la clause : elle remet le calcul à zéro à chaque nouvelle valeur de partition.

Théorie : `Theorie_04_SQL.md`, sections 4 et 8.

P10. (avancé)

```sql
SELECT name, listprice, productsubcategoryid,
       MAX(listprice) OVER (PARTITION BY productsubcategoryid) AS prix_max_sous_cat,
       (listprice - MAX(listprice) OVER (PARTITION BY productsubcategoryid))
         / NULLIF(MAX(listprice) OVER (PARTITION BY productsubcategoryid), 0) AS ecart_pct
FROM production.product
WHERE listprice > 0 AND productsubcategoryid IS NOT NULL
ORDER BY productsubcategoryid, listprice DESC;
```

L'intérêt de l'exercice est là : sans fenêtre, il aurait fallu une sous-requête d'agrégat par sous-catégorie, puis une jointure pour la rapprocher de chaque produit. La fenêtre fait le même travail en une passe, et c'est exactement le cas d'usage qui justifie l'outil.

Théorie : `Theorie_04_SQL.md`, section 5.

P11. (avancé)

```sql
SELECT productsubcategoryid, AVG(listprice) AS mediane
FROM (
    SELECT productsubcategoryid, listprice,
           ROW_NUMBER() OVER (PARTITION BY productsubcategoryid ORDER BY listprice) AS rang,
           COUNT(*)     OVER (PARTITION BY productsubcategoryid)                     AS n
    FROM production.product
    WHERE listprice > 0 AND productsubcategoryid IS NOT NULL
) AS q
WHERE rang IN ((n + 1) / 2, (n + 2) / 2)
GROUP BY productsubcategoryid
ORDER BY productsubcategoryid;
```

Attendu : 37 lignes, une par sous-catégorie qui a des produits tarifés.

© 2026 iAlexMG. Tous droits réservés.

La mécanique tient dans la ligne du `WHERE`, et elle mérite d'être lue lentement. `ROW_NUMBER` numérote les prix d'une sous-catégorie du plus bas au plus haut ; `COUNT(*) OVER (PARTITION BY …)` donne, sur chaque ligne, le compte de son groupe. La division est entière : quand `n` est impair, `(n + 1) / 2` et `(n + 2) / 2` désignent la même ligne, celle du milieu, et la moyenne d'une seule valeur est cette valeur. Quand `n` est pair, elles désignent les deux lignes du milieu, et la moyenne des deux est la médiane. Un seul `WHERE` couvre donc les deux cas.

La faute à ne pas faire est celle qui vient naturellement : filtrer sur `PERCENT_RANK() >= 0.5` et prendre les lignes restantes. `PERCENT_RANK` distribue les rangs sur un axe de 0 à 1, mais il ne garde pas une ligne par groupe : le filtre laisse passer toute la moitié supérieure, et le `DISTINCT` qu'on ajoute pour rattraper le coup rend autant de lignes qu'il y a de prix distincts au-dessus de la médiane. Mesuré sur cette base : 48 lignes pour 23 sous-catégories, au lieu de 37 médianes.

La fonction qui l'aurait fait en une ligne est `PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY listprice)`, disponible sur les deux moteurs. C'est aussi la preuve du corrigé : la requête ci-dessus lui est égale à la quatrième décimale sur les 37 sous-catégories. L'exercice demande l'autre chemin parce qu'il oblige à manipuler un rang et un compte dans la même passe, ce que la fonction d'ordre supérieur cache. En production, on écrit la version courte.

Théorie : `Theorie_04_SQL.md`, sections 3 et 6.

P12. `(avancé)`

```sql
SELECT annee, mois, ventes,
       AVG(ventes) OVER (ORDER BY annee, mois
                         ROWS BETWEEN 3 PRECEDING AND 1 PRECEDING) AS moy_3_avant,
       AVG(ventes) OVER (ORDER BY annee, mois
                         ROWS BETWEEN 1 FOLLOWING AND 3 FOLLOWING)  AS moy_3_apres
FROM mensuel
ORDER BY annee, mois;
```

Trois lignes sont touchées de chaque côté, pas deux. `moy_3_avant` vaut `NULL` sur la première ligne, puis moyenne un seul mois sur la deuxième et deux sur la troisième ; `moy_3_apres` fait le même chemin à l'envers sur les trois dernières, deux mois, un mois, puis `NULL`. Le cadre ne provoque pas d'erreur quand il déborde, il se contente de ce qu'il trouve, ce qui est commode et qu'il faut savoir, sous peine de comparer une moyenne de trois mois à une moyenne d'un seul.

Le dernier mois aggrave le cas. Sur l'entrepôt, janvier 2014 ne pèse que 45 694,72 $, une quarantaine de fois moins que décembre, alors qu'il a des ventes sur 28 jours ; la `moy_3_apres` de décembre 2013 n'est que ce seul chiffre. Sur le transactionnel, juin 2025 joue le même rôle, 47 491,55 $ sur 29 jours. Mesuré des deux côtés. Une bonne réponse le remarque et propose d'afficher le nombre de mois moyennés à côté de la moyenne, par un `COUNT(*)` sur le même cadre, et de regarder le mois du bord avant de se fier à ce qu'il tire.

Théorie : `Theorie_04_SQL.md`, section 6.

Diagnostics

E1. Sans cadre explicite, un `OVER (ORDER BY …)` applique le cadre par défaut, qui va du début de la partition jusqu'à la ligne courante. La colonne calculée est donc la moyenne de tous les mois depuis le premier, une moyenne cumulative, qui se lisse de plus en plus et converge lentement. Elle ne dit rien des trois derniers mois.

© 2026 iAlexMG. Tous droits réservés.

Le mécanisme en cause est le cadre par défaut, et c'est le piège le plus fréquent du fenêtrage, parce que la requête ne produit ni erreur ni valeur absurde : elle produit une colonne plausible.

```
AVG(ventes) OVER (ORDER BY annee, mois ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)
```

Théorie : `Theorie_04_SQL.md`, section 6.

E2. Il filtre sur une colonne calculée par une fenêtre, dans le `WHERE`. Or le `WHERE` s'exécute avant les fenêtres : au moment où il est évalué, `rang` n'existe pas. Le message est trompeur parce qu'il parle d'une colonne inexistante, ce qui suggère une faute de frappe, alors que le problème est un problème d'ordre d'exécution. PostgreSQL gère parfaitement les fonctions de fenêtrage, il refuse simplement qu'on filtre dessus trop tôt.

```
-- Sous-requete
SELECT name, listprice
FROM (
    SELECT name, listprice,
           ROW_NUMBER() OVER (PARTITION BY productsubcategoryid
                              ORDER BY listprice DESC) AS rang
    FROM production.product
) AS c
WHERE rang <= 3;

-- CTE (Cours 05), meme chose en plus lisible
WITH classe AS (
    SELECT name, listprice,
           ROW_NUMBER() OVER (PARTITION BY productsubcategoryid
                              ORDER BY listprice DESC) AS rang
    FROM production.product
)
SELECT name, listprice FROM classe WHERE rang <= 3;
```

Théorie : `Theorie_04_SQL.md`, sections 2 et 7.

E3. Deux fautes distinctes, et il faut les voir séparément.

La première est le cadre. `SUM(ventes) OVER (ORDER BY annee)` applique le cadre par défaut, `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`, donc un cumul depuis le début, et non le total de l'année.

Le détail qui compte est dans le mot `RANGE`, et c'est lui qui explique les symptômes. Un cadre en `RANGE` ne s'arrête pas à la ligne courante : il va jusqu'au bout des pairs, c'est-à-dire de toutes les lignes qui ont la même valeur de tri. Ici le tri porte sur `annee`, donc les douze mois d'une année sont pairs entre eux, et reçoivent tous le même dénominateur, le cumul arrêté à la fin de leur année. Mesuré : 14 561 051,59 pour les mois de 2022, 46 165 973,54 pour ceux de 2023, et ainsi de suite.

© 2026 iAlexMG. Tous droits réservés.

De là les deux symptômes. La première année tombe juste, à 1,0000, parce que le cumul arrêté à la fin de la première année est exactement le total de cette année. Les suivantes s'additionnent à moins que 1, et de moins en moins, parce que leur dénominateur porte aussi toutes les années d'avant : 0,6846, 0,4861, 0,1821. Le premier mois d'une année n'a rien de particulier, contrairement à ce qu'on croirait : en 2022, il pèse 0,0356 pour une moyenne de 0,1250. Si le cadre avait été en `ROWS`, le dénominateur changerait à chaque ligne et aucune année ne tomberait juste. C'est la différence entre `ROWS` et `RANGE` de la section 6, et c'est le seul exercice du cours où elle change la réponse.

La seconde est la partition. Pour un total annuel, il faut `PARTITION BY annee`, pas `ORDER BY annee`. L'auteur a confondu les deux clauses, ce qui est fréquent parce qu'elles portent le même nom de colonne.

```sql
SELECT annee, mois, ventes,
       ventes / NULLIF(SUM(ventes) OVER (PARTITION BY annee), 0) AS part_annee
FROM mensuel;
```

On en profite pour protéger la division, ce que l'original ne faisait pas non plus.

Théorie : `Theorie_04_SQL.md`, sections 2 et 6.

Questions à développer

A. Une fonction de fenêtre ne réduit rien : elle ajoute une colonne à des lignes qui restent toutes là. Un `GROUP BY`, lui, remplace n lignes par une, et tout ce qui n'est pas dans le regroupement ou dans un agrégat disparaît. C'est cette conservation qui rend le fenêtrage indispensable au reporting, parce qu'un rapport veut presque toujours le détail et le contexte sur la même ligne.

L'exemple qui le montre : afficher chaque vente avec la part qu'elle représente dans le total de son produit. Avec `GROUP BY` seul, il faut une première requête qui calcule les totaux par produit, une seconde qui lit les ventes, et une jointure entre les deux, donc deux passes sur les données et un risque d'incohérence si les deux ne sont pas exécutées sur le même instantané. Avec une fenêtre, c'est une passe et une ligne de SQL.

Théorie : `Theorie_04_SQL.md`, section 1.

B. Sur la syntaxe, rien ne l'attend : les fenêtres, les cadres, les fonctions de classement sont identiques. La seule différence de dialecte notable est `QUALIFY`, absent de PostgreSQL, qu'on remplace par une sous-requête ou une CTE.

Sur le schéma, tout l'attend. Le contexte d'une vente est éclaté dans plusieurs tables, donc les jointures se multiplient. Il n'y a pas de dimension temps, donc l'année et le mois se calculent à la main sur un horodatage. Les libellés vivent dans des tables de référence séparées. Une requête qui tenait en dix lignes sur l'entrepôt en fait vingt-cinq, sans que le calcul soit plus difficile.

Sur la performance, le renversement est complet, et il vaut la peine d'être dit exactement. En OLTP, on optimise en donnant au moteur un index qui porte déjà l'ordre voulu, et en filtrant tôt pour toucher peu de lignes : sur `sales.salesorderdetail`, un index sur la partition et l'ordre de la fenêtre fait disparaître du plan un tri que le moteur déversait sur disque. En OLAP, le même geste ne donne rien, mesuré sur DuckDB, où le plan et le temps restent les mêmes avec ou sans index : on assume de balayer, et c'est le stockage en colonnes qui rend le balayage bon marché. Le réflexe qu'on ramène d'un monde est donc souvent

© 2026 iAlexMG. Tous droits réservés.

inutile dans l'autre, et parfois nuisible : créer un index sur un entrepôt coûte de l'écriture sans rien rendre, il n'y en a même pas sur Snowflake d'après sa documentation, et balayer 40 millions de lignes sur la base transactionnelle de production est une façon de gêner les clients de l'entreprise.

Théorie : `Theorie_04_SQL.md`, section 8.

C. Pour `QUALIFY` : il supprime un niveau d'imbrication, donc une source d'erreur et de fatigue à la lecture. Un top N par groupe s'écrit en trois lignes lisibles au lieu de sept. Sur une base de code qui vit sur un moteur qui l'a, c'est un gain réel et quotidien, et se priver d'une facilité du moteur qu'on a choisi n'a pas de sens.

L'argument sérieux qu'on lui oppose : il n'est pas dans la norme. Une équipe qui écrit du SQL destiné à tourner sur plusieurs moteurs, ou qui pourrait migrer, se crée une dette à chaque `QUALIFY`. Le coût ne se paie pas à l'écriture, il se paie le jour de la migration, en une centaine de requêtes à reprendre.

La ligne de partage n'est pas celle qu'on suppose, et c'est ce que les deux labos de ce cours rendent tangible. On imagine volontiers `QUALIFY` comme le luxe d'une grande plateforme, réservé à qui paie un entrepôt cloud. La réalité est que DuckDB, un moteur qui tient dans un fichier sur votre poste, l'a, et que PostgreSQL, un moteur autrement plus gros et plus ancien, ne l'a pas. Le clivage est celui de la vocation : les moteurs analytiques ont adopté ce raccourci parce que le top N par groupe y est une question de tous les jours, les transactionnels ne l'ont pas parce qu'elle y est rare. Se tromper là-dessus mène à une mauvaise prévision de migration, ce qui est exactement le genre d'erreur que ce cours cherche à éviter.

La règle que j'adopterais : `QUALIFY` est autorisé dans le code spécifique à un moteur analytique qu'on a choisi et qu'on n'a pas l'intention de quitter, et proscrit dans tout ce qui est censé rester portable, typiquement les modèles d'une couche partagée ou les requêtes livrées à un client. Et la règle s'écrit quelque part, parce qu'une convention qui n'est pas écrite se perd au premier renfort dans l'équipe.

Théorie : `Theorie_04_SQL.md`, section 7.

Mises en situation

M1. Épicerie Bellefeuille.

La requête s'organise en trois niveaux. Le premier agrège les lignes de ticket par succursale, par produit et par semaine, ce qui donne les ventes hebdomadaires. Le deuxième pose deux fenêtres sur ce résultat : un `ROW_NUMBER() OVER (PARTITION BY succursale, semaine ORDER BY ventes DESC)` pour le rang, et un `ventes / SUM(ventes) OVER (PARTITION BY succursale, semaine)` pour la part. Le troisième filtre sur `rang <= 5`, et il doit être un niveau séparé, puisque PostgreSQL n'a pas `QUALIFY`.

La progression de rang est le point difficile pour une raison précise : elle demande de comparer le rang de cette semaine au rang de la semaine précédente, or ce rang précédent est lui-même le résultat d'une fenêtre. On ne peut pas imbriquer un `LAG` sur une colonne calculée par une autre fenêtre dans la même clause `SELECT`. Il faut donc calculer les rangs de toutes les semaines dans un niveau, puis appliquer `LAG(rang) OVER (PARTITION BY succursale, produit ORDER BY semaine)` au niveau suivant. Le piège corollaire : un produit absent du palmarès la semaine précédente n'a pas de rang antérieur, et `LAG` renverra `NULL`. Il faut décider ce qu'on affiche, un tiret plutôt qu'une fausse progression, et surtout ne pas filtrer sur `rang <= 5` avant d'avoir calculé le décalage, sinon on perd justement l'information des entrants et des sortants.

© 2026 iAlexMG. Tous droits réservés.

Sur la question qui décide de tout : cet écran ne doit pas être servi par la base transactionnelle. Trois raisons, dans l'ordre. La requête balaie plusieurs semaines de tickets sur douze succursales, donc elle est lourde, et la base transactionnelle est celle qui encaisse les passages en caisse en temps réel ; une requête analytique lourde y ralentit les clients au comptoir. Le besoin est hebdomadaire, donc la fraîcheur à la seconde n'apporte rien. Et le calcul devra être répété et comparé dans le temps, ce qui appelle une table de résultats stable plutôt qu'un recalcul.

Ce qui pourrait faire changer d'avis : une entreprise sans entrepôt et sans les moyens d'en monter un. Dans ce cas, la réponse honnête est une réplique en lecture de la base transactionnelle, pas la base de production elle-même.

Théorie : `Theorie_04_SQL.md`, sections 3, 4, 5 et 7.

M2. Clinique vétérinaire Deux-Montagnes.

Le délai entre deux visites est un `LAG` sur la date, partitionné par animal : `consultation - LAG(consultation) OVER (PARTITION BY animal_id ORDER BY consultation)`. Un seul niveau suffit pour cette première demande.

Le délai moyen par espèce demande un second niveau, parce qu'il faut agréger les délais qu'on vient de calculer. On enveloppe le premier résultat et on fait `AVG(delai) GROUP BY espece`, ou bien `AVG(delai) OVER (PARTITION BY espece)` si l'on veut garder le détail à côté.

La première visite de chaque animal n'a pas de précédente, donc `delai` vaut `NULL`. C'est le comportement voulu, et il faut y prendre garde à deux endroits : `AVG` ignore les `NULL`, ce qui est heureux, mais un `COUNT(*)` compterait ces lignes, ce qui fausserait toute moyenne calculée à la main. Les animaux vus une seule fois n'ont donc aucun délai du tout ; ils ne doivent pas être écartés pour autant, car ce sont souvent eux qui intéressent la clinique. On les traite dans une catégorie à part plutôt que de les laisser disparaître d'un `WHERE delai IS NOT NULL`.

La dernière demande ne peut pas se traiter dans la même fenêtre pour une raison de nature : elle compare une durée en cours, entre la dernière visite et aujourd'hui, à une statistique agrégée, le délai habituel de l'animal. La première n'est pas un délai entre deux lignes, c'est un délai entre une ligne et le présent, donc `LAG` ne peut rien pour elle. Il faut un niveau qui isole la dernière visite de chaque animal, un autre qui calcule sa moyenne historique, et une comparaison entre les deux.

Théorie : `Theorie_04_SQL.md`, sections 4 et 5.

M3. `(avancé)` Coopérative forestière Abitibi.

Les deux requêtes ont la même ossature : agréger le volume par coupe, région et mois, puis poser deux fenêtres, `volume / SUM(volume) OVER (PARTITION BY region, mois)` pour la part et `SUM(volume) OVER (PARTITION BY region ORDER BY mois ROWS BETWEEN 11 PRECEDING AND CURRENT ROW)` pour le cumul annuel glissant.

Attention au grain de cette seconde fenêtre, et c'est le point que le correcteur regarde. Au grain coupe, région et mois, une région porte autant de lignes par mois qu'elle a de coupes : `ROWS BETWEEN 11 PRECEDING` y compte alors douze **lignes**, pas douze mois, et le cumul annoncé annuel ne l'est pas. Deux écritures s'en sortent. Soit on pose la fenêtre sur un résultat déjà agrégé au grain région et mois, puis on rejoint ce cumul aux lignes de détail ; soit on garde le grain fin et on écrit un cadre en durée, `RANGE BETWEEN INTERVAL 11 MONTH PRECEDING AND CURRENT ROW`, qui raisonne en valeurs de date et non en lignes. La première

© 2026 iAlexMG. Tous droits réservés.

voyage partout, la seconde est plus courte et passe sur nos deux terrains. Ce qui diffère est en amont : sur l'entrepôt, une jointure du fait vers `dim_date` et `dim_region` ; sur le transactionnel, une reconstitution à partir des livraisons, des chantiers et de la table des régions, plus un `EXTRACT` sur l'horodatage.

Axe	Voie OLTP	Voie OLAP
Nombre de jointures	quatre ou cinq, selon la profondeur du référentiel	deux
Lisibilité	requête longue, imbriquée, difficile à relire à six mois	courte, le vocabulaire est celui du métier
Fraîcheur	à la seconde	celle de la dernière alimentation, ici la nuit précédente
Risque pour le système	réel, on balaie 40 millions de lignes sur la base qui fait tourner l'exploitation	nul, l'entrepôt est fait pour ça

L'axe qui joue contre l'entrepôt est la fraîcheur, et il ne faut pas l'escamoter. Si la coopérative veut savoir ce qui a été livré ce matin, l'entrepôt ne le sait pas encore et ne le saura qu'à la prochaine alimentation. Répondre « on rafraîchira plus souvent » est une réponse de façade tant qu'on n'a pas chiffré ce que coûte une alimentation horaire.

Je mettrais en production la voie OLAP, parce que le rapport demandé est mensuel avec un cumul annuel : un décalage d'une nuit n'a aucune conséquence sur une lecture à cette maille, alors que la charge imposée à la base d'exploitation en aurait. Je changerais d'avis dans deux cas seulement. Si le besoin devenait opérationnel, par exemple suivre les livraisons du jour pour affecter des camions, la fraîcheur deviendrait la contrainte dominante et la question ne serait plus la même. Et si l'entrepôt s'avérait ne pas contenir une donnée que le système d'exploitation est seul à avoir, auquel cas le vrai correctif serait de l'ajouter à l'alimentation, pas de contourner l'entrepôt.

Théorie : `Theorie_04_SQL.md`, sections 5, 6 et 8.

© 2026 iAlexMG. Tous droits réservés.